

Experiment #08

Creating Logical C++ codes using Function Overloading concepts

Objective

Objective of this lab session is to reinforce some basic programming concepts regarding to function overloading in C++.

Software Tools

Dev C++

Theory

Function

A function is a subprogram that acts on data and often returns a value. A program written with numerous functions is easier to maintain, update and debug than one very long program. By programming in a modular (functional) fashion, several programmers can work independently on separate functions which can be assembled at a later date to create the entire project. Each function has its own name. When the function is finished, execution returns to the area of the program code from which it was called, and the program continues on to the next line of code.

Steps for creating user-defined functions

Declare the function

The declaration, called the **FUNCTION PROTOTYPE**, informs the compiler about the functions to be used in a program, the argument they take and the type of value they return.

Define the function

The function definition tells the compiler what task the function will be performing. The function prototype and the function definition must be same on the return type, the name, and the parameters. The only difference between the function prototype and the function header is a semicolon.

The function definition consists of the function header and its body. The header is EXACTLY like the function prototype, EXCEPT that it contains NO terminating semicolon.

//Prototyping, defining and calling a function

```
#include <iostream>
using namespace std;
void func(); // prototype the function
int main()
{
    func( ); // function call
    cout<< "\t\tBjarne Stroustrup\n";
    func( ); // function call
    return 0;
}
```

```
    // function definition
void func()
{
    int count; // declaring a LOCAL variable
    for(count = 1; count <=65; count++)
        cout<< "*";
    cout<<endl;
}
```

Argument to a function

Sometimes the calling function supplies some values to the called function. These are known as parameters. The variables which supply the values to a calling function called **actual parameters**. The variable which receive the value from called statement are termed **formal parameters**.

Passing int type argument to a function

```
#include<iostream>
using namespace std;
void age(int);
int main()
{
    int a;
    cout<<"enter two ages:";
    cin>>a>>b;
    age(a);
    return 0;
}
void area(int b)
{
    cout<< "sum of ages is"<<b+b<<"\n";
}
```

Passing float type argument to a function

```
#include<iostream>
using namespace std;
void area(float);
int main()
{
    float radius;
    cin>>radius;
    area(radius);
    return 0;
}
```

```
void area(float r)
{
```

Department of Electrical Engineering
HITEC University Taxila

2
EC-112 Object Oriented Programming

```
    cout<< "the area of the circle is"<<3.14*r*r<<"\n";
}
```

Here radius is called **actual parameter** and r is called **formal parameter**.

Passing an array to a function

```
#include <iostream>
using namespace std;

void func1 (int arg[]) {
    for (int n = 0; n < 5; n++) {
        cout << arg[n] << " ";
        cout << "\n";
    }
}

int main ()
{
    int firstarray[] = {5, 10, 15};
    int secondarray[] = {2, 4, 6, 8, 10};
    func1 (firstarray);
    func1 (secondarray);

    return 0;
}
```

Passing structure to a function

```
#include <iostream>
using namespace std;

struct Person
{
    char name[50];
    int age;
    float salary;
};

void displayData(Person); // Function declaration
```

```
int main()
{
    Person p;
    Department of Electrical Engineering
    HITEC University Taxila
```

3
EC-112 Object Oriented Programming

```
    cout << "Enter Full name: ";
    cin << p.name;
    cout << "Enter age: ";
    cin >> p.age;
    cout << "Enter salary: ";
    cin >> p.salary;

    // Function call with structure variable as an argument
    displayData(p);

    return 0;
}
```

```
void displayData(Person p)
{
    cout << "\nDisplaying Information." << endl;
    cout << "Name: " << p.name << endl;
    cout << "Age: " << p.age << endl;
    cout << "Salary: " << p.salary;
}
```

Return type of a function

// Example program

```
#include <iostream>
using namespace std;
```

```
int timesTwo(int num); // function prototype
```

```

int main()
{
    int number, response;
    cout<<"Please enter a number:";
    cin>>number;
    response = timesTwo(number); //function call

```

Department of Electrical Engineering
HITEC University Taxila

4
EC-112 Object Oriented Programming

```

    cout<< "The answer is "<<response;
    return 0;
}

```

```

//timesTwo function
int timesTwo (int num)
{
    int answer; //local variable
    answer = 2 * num;
    return (answer);
}

```

Function with default arguments

C++ allows to call a function without specifying all its arguments. In such cases, the function assigns a default value to a parameter which does not have a matching arguments in the function call. Default values are specified when the function is declared. The compiler knows from the prototype how many arguments a function uses for calling.

Example

```
float result(int marks1, int marks2, int marks3=75);
```

a subsequent function call

```
average = result(60,70);
```

passes the value 60 to marks1, 70 to marks2 and lets the function use default value of 75 for marks3.

The function call

```
average = result(60,70,80);
```

passes the value 80 to marks3.

Global variable and local variable

Local Variable: A variable declared within the body of a function will be evaluated only within the function. The portion of the program in which a variable is retained in memory is known as the **scope of the variable**. The scope of the local variable is a function where it is defined. A variable may be local to function or compound statement.

Global variable: A variable that is declared outside any function is known as a global variable. The scope of such a variable extends till the end of the program. These variables are available to all functions which follow their declaration. So it should be defined at the beginning, before any function is defined.

Unary scope resolution operator (::) It is possible to declare local and global variables of the same name. C++ provides the **unary scope resolution operator (::)** to access a global variable when a local variable of the same name is in scope. A global variable can be accessed directly without the unary scope resolution operator if the name of the global variable is not the same as

that of a local variable in scope.

Function overloading

Function overloading allows you to use the same name for different functions. Function overloading is usually used to enhance the readability of the program. If you have to perform one single operation but with different number or types of arguments, then you can simply overload the function.

Ways to overload a function

1. By changing number of Arguments.

Department of Electrical Engineering
HITEC University Taxila

5

EC-112 Object Oriented Programming

2. By having different types of argument.

Different number of arguments

In this type of function overloading, we define two functions with same names but different number of parameters of the same type. For example, in the below mentioned program we have made two sum() functions to return sum of two and three integers.

```
int sum (int x, int y)
{
    cout << x+y;
}
```

```
int sum(int x, int y, int z)
{
    cout << x+y+z;
}
```

Here sum() function is overloaded, to have two and three arguments. Which sum() function will be called, depends on the number of arguments.

```
int main()
{
    sum (10,20); // sum() with 2 parameter will be called

    sum(10,20,30); //sum() with 3 parameter will be called
}
```

Different data type of arguments

In this type of overloading, we define two or more functions with same name and same number of parameters, but the type of parameter is different. For example in this program, we have two sum() function, first one gets two integer arguments and second one gets two double arguments.

```
int sum(int x,int y)
{
    cout<< x+y;
}
```

```
double sum(double x,double y)
{
    cout << x+y;
```

```

}

int main()
{
    sum (10,20);
    sum(10.5,20.5);
}

```

Lab Tasks

Department of Electrical Engineering
HITEC University Taxila

6
EC-112 Object Oriented Programming

1. Create and Implement program in C++ to define a class. Define a function multiplication in it and perform multiplication of 2 variables. Implement function overloading by using different type of variables/arguments. Pass values to data variables in main. Also print the multiplication results.
2. Create and Implement program in C++ to define a class. Define a function multiplication in it and perform multiplication of n variables. Implement function overloading by using different number of variables/arguments. Pass values to data variables in main. Also print the multiplication results.
3. Create and Implement program in C++ by using function overloading to calculate absolute value of a number of type integer, float and double. Also print the original value as well as the absolute value using the function.

Conclusion

Lab Instructor

Kaynat Rana

Department of Electrical Engineering
HITEC University Taxila